FlexFit Studio: Data Model Design

Salesforce Administrator portfolio project: Phase 1 (Data Model)

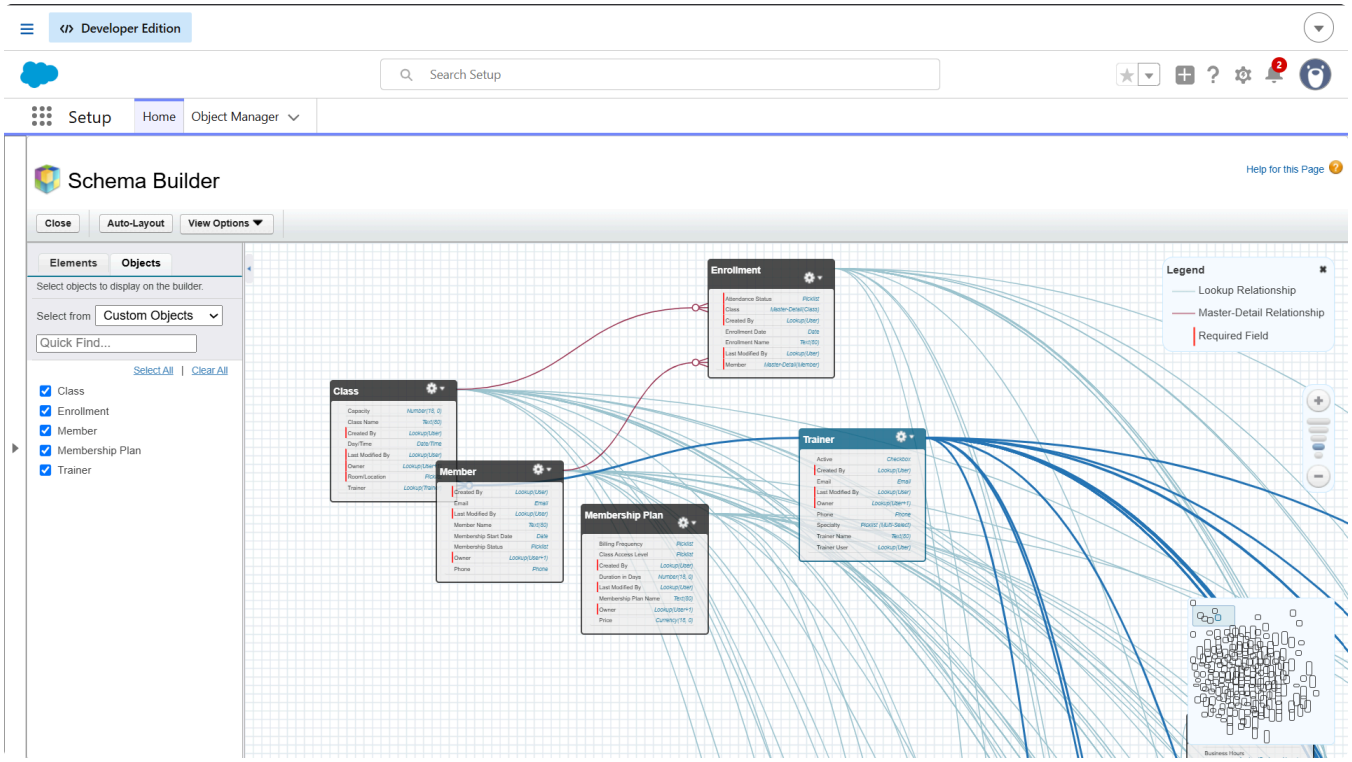
Overview

FlexFit Studio is a fictional gym management org I built in a Salesforce Developer Edition org. Phase 1 covers the data model: five custom objects, the fields on each, and the relationships between them. The goal was a model that matches how a real studio operates (members, trainers, classes, and who attends what) while keeping data clean and reportable.

The Business Problem

A gym needs to track who its members are, which membership they hold, which trainers teach which classes, and which members attend which classes. A flat spreadsheet breaks down quickly: a member can attend many classes, a class has many members, and each class has one trainer. The data model needs to represent those relationships directly.

The Data Model



Objects and key fields

- **Member:** email, phone, membership start date, and a Membership Status picklist (Active, Expired, Frozen, Cancelled).
- **Membership Plan:** plan name, price (currency), billing frequency (picklist), class access level (picklist), and duration in days.
- **Trainer:** specialty (multi-select picklist), email, phone, an Active checkbox, and a Trainer User lookup to a Salesforce user.
- **Class:** class name, day and time, capacity, a Room/Location picklist, and a Trainer lookup.
- **Enrollment:** the junction object between Member and Class, with an enrollment date and an attendance status picklist (Registered, Attended, No-Show, Cancelled).

Relationships

- **Class to Trainer: Lookup.** A class can exist before a trainer is confirmed, so it should not depend on one. The delete behavior is set to "Don't allow deletion," so a trainer with classes cannot be deleted by accident, which protects class history.
- **Enrollment to Member and Class: Master-Detail (both).** An enrollment has no meaning without both a member and a class, so it cannot exist without them. Master-Detail also allows roll-up summary fields later (for example, enrollments counted per class to show spots filled) and makes enrollment access follow the parent records.
- **Trainer to User: Lookup.** This links a Trainer record to a real Salesforce user. It was added later, in Phase 3, so automation could connect a class to the trainer's user account.

Design Decisions

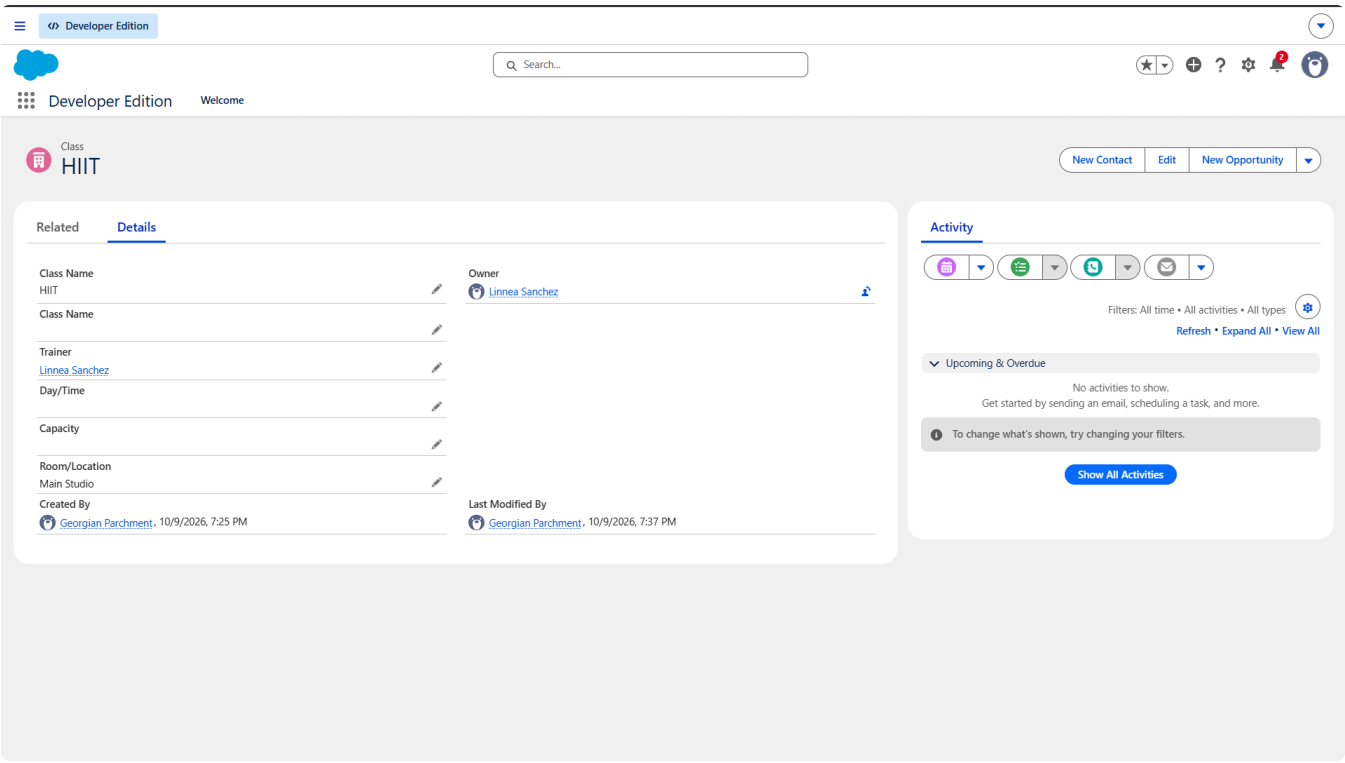
Junction object. Members and classes are many-to-many: a member takes many classes, and a class has many members. A junction object (Enrollment) models that cleanly. Members who only use the general gym simply have zero enrollments, which is valid and easy to report on.

Picklists instead of free text. Fields like Membership Status and Room/Location are picklists so that values stay consistent. With free text, "Room 1," "room one," and "Studio A" would all be different values and break reports and filters.

Lookup versus Master-Detail. I used a Lookup where the child record can reasonably exist on its own (Class and Trainer) and Master-Detail where it cannot (Enrollment). Choosing the relationship type by asking "can this record exist without its parent?" keeps the model honest.

Result

The model supports the reporting a studio would need: active versus expired members, classes by trainer, and attendance by class. It also gives the later phases a foundation, including security by role, automation, and dashboards.



Skills Demonstrated

- Custom object and field design
- Choosing Lookup versus Master-Detail relationships, and setting delete behavior
- Junction objects for many-to-many relationships
- Using picklists to protect data quality
- Creating tabs so objects are reachable in the UI

What I Would Add Next

- A validation rule blocking enrollment when a member's status is Expired or Cancelled
- Roll-up summary fields to show spots filled per class
- Reports and dashboards on top of the model